AMIRIS - Install and Live Demo Guide

Everything needed to install, run, and demonstrate the AMIRIS scenarios this project has already produced

Muideen Oladayo Ajiboye | 30 July 2026

Part A - One-Time Setup

1. **Confirm Java is installed**

   AMIRIS's simulation engine is a Java program. Open PowerShell and run the check below. Any reasonably recent Java version works (this machine has a well-above-minimum version already).

```
java -version
```

2. **Confirm the Python environment exists**

   This project already has a ready-made virtual environment with amirispy (the command-line tool) and fameio (the scenario compiler/extractor) installed, at:

```
C:\Users\MuideenOA\Desktop\PyTut\Amiris\py313env
```

If demonstrating on a different machine without this folder, create it fresh instead:

```
python -m venv py313env
.\py313env\Scripts\Activate.ps1
pip install amirispy fameio
```

3. **Confirm the engine file exists**

   The actual simulation engine, a single Java file, sits in the project root:

```
amiris-core_4.1.2-jar-with-dependencies.jar
```

Part B - Every Time You Demo

1. **Open PowerShell in the project folder**

```
cd C:\Users\MuideenOA\Desktop\PyTut\Amiris
```

2. **Activate the Python environment**

   You'll see the prompt change to show (py313env) once this works.

```
.\py313env\Scripts\Activate.ps1
```

3. **Run a scenario**

   This is the one command that does everything: compiles the scenario, runs the full year hour-by-hour in the Java engine, then extracts the results back into readable CSVs. Example using the Germany2019 backtest:

```
amiris run --log info `
  --scenario "examples\backtest\Germany2019\scenario.yaml" `
  --jar "amiris-core_4.1.2-jar-with-dependencies.jar" `
  --output "result_Germany2019_demo"
```

Gotcha - order matters:

--log must come BEFORE the word 'run', not after. Putting it after the subcommand fails silently with exit code 2 - this has actually happened once already in this project.

A full year takes anywhere from under a minute to a few minutes depending on the machine. To demo a different

year, just change the folder name in --scenario and give --output a fresh name so you don't overwrite an existing result.

4. **Show the result**

   Open the output folder. The file that matters most for a demo is:

```
result_Germany2019_demo\DayAheadMarketSingleZone.csv
```

Its ElectricityPriceInEURperMWH column is AMIRIS's simulated hourly electricity price for the whole year - 8,760 rows. This is the number the entire thesis is built around. metadata.json in the same folder explains what every column in every CSV means, useful if asked to explain a specific file live.

Scenario Inventory - What's Actually Ready Right Now

Checked directly against the project folder before writing this guide, so this reflects the real current state, not what was originally planned.

Scenario	Status	Notes
Germany 2015-2019	Ready - already run	Results exist in result_Germany2015 .. result_Germany2019. Safest choice to demo live.
Austria 2019	Ready - not yet run	Full scenario folder present (examples\backtest\Austria2019), never actually executed in this project yet.
Germany 2023	NOT ready	Only the raw input timeseries (load, outages, fuel prices, generation profiles) have been sourced from ENTSO-E/FRED so far. scenario.yaml, schema.yaml, and the agents/contracts folders that actually define the plant fleet do not exist yet - this scenario cannot be run yet.
Germany 2027 (Brainpool)	Not started	Waiting on the data from the supervisor before scenario-building begins.

Recommendation for the physical demo:

Run Germany2019 live - it is fully built, has already produced validated results once, and is the scenario this thesis's core findings (the 2018/2019 root-cause analysis) are grounded in.

Troubleshooting Quick List

- 'exit code 2' immediately on run -> almost always the --log flag placed after run instead of before it.
- Output folder error / file appears locked -> something else (Explorer preview, Excel) has a file in that folder open; close it, or give --output a new folder name.
- A plotting script fails mentioning Tk -> matplotlib is trying to open an interactive window; if only saving a PNG is needed, this is a backend setting, not a real failure - said script simply is not present in the project folder right now, this note is here in case it gets rebuilt before the meeting.

Does AMIRIS Auto-Update From New Input Data, or Does the YAML Need Manual Edit

Short answer: it depends on the field, and AMIRIS never reaches out and updates anything on its own - it only ever reads whatever is sitting in the scenario folder at the moment 'amiris run' is executed. There are exactly two situations.

Situation 1: the field already points to a file (automatic pickup)

Several fields in Conventionals.yaml and RenewablesAndPolicy.yaml are not numbers at all - they are a path to a CSV, e.g. Nuclear's real entry:

```
OutageFactor: "./timeseries/nuclear_outage.csv"
MustRunFactor: "./timeseries/nuclear_must_run.csv"
```

For these, no YAML edit is needed at all. Overwrite nuclear_outage.csv with the 2027 outage data, keep the exact same filename in the exact same timeseries folder, and the next 'amiris run' automatically picks up the new numbers. This also covers hard coal/gas/oil fuel prices, the CO2 price, and every renewable YieldProfile - all file references already, all just need the underlying CSV replaced.

Situation 2: the field is a hardcoded number in the YAML (manual edit required)

Other fields are typed directly into the YAML as a plain number, not a file reference. Nuclear again, same file:

```
InstalledPowerInMW: 9524.0
Efficiency:
  Minimal: 0.330
  Maximal: 0.331
BlockSizeInMW: 900.0
OpexVarInEURperMWH: 0.5
```

AMIRIS has no mechanism to notice that Germany's real 2027 nuclear capacity is different (it's not - the last plants closed in 2023, but this illustrates the mechanic) or that efficiency assumptions have shifted. These have to be opened and typed in by hand - or by a small script that reads the new figure and rewrites the YAML value, the same approach already used for the 2023 timeseries pulls.

Field type	Example	To update for 2027
File-referenced (time series)	OutageFactor, MustRunFactor, FuelPrices (coal/gas/oil), Co2Prices, YieldProfile	Just replace the CSV file, same name, same folder. No YAML edit.
Hardcoded scalar	InstalledPowerInMW, Efficiency Min/Max, BlockSizeInMW, OpexVarInEURperMWH, SpecificCo2EmissionsInTperMWH, FIT/LCOE rates, storage capacities	Must manually edit the YAML (or run a small script that does it) - AMIRIS will not pick this up on its own.

Live proof of this, sitting in the project right now:

The Germany 2023 build shows exactly this split in progress. Its timeseries folder is fully populated with real ENTSO-E/FRED data - the file-referenced side is done. But its agents\Conventionals.yaml, RenewablesAndPolicy.yaml, and Storage.yaml do not exist yet - the hardcoded-scalar side (fleet capacities, efficiencies, subsidy rates for 2023) still has to be authored by hand, which is exactly why 2023 cannot be run yet even though its raw data is ready.